

VETRA — Site Documentation

This document explains what VETRA is, how the codebase is organized, what every page and feature does, how the pieces fit together, and what to do next if you want to turn this prototype into a production product. It covers the public marketing site, the buyer (customer) app, the vendor app, and the admin console.

1. What VETRA is

VETRA is a marketplace concept for buyers ("customers") and sellers ("vendors") in Nigeria, plus an internal **admin console** for VETRA staff to monitor and moderate the platform. Right now it is a **front-end prototype**: there is no server, no database, and no real authentication. Every page is a static HTML file styled with plain CSS and made interactive with small, dependency-free JavaScript files. Where a page needs to "remember" something between visits (currently only the admin console), it uses the browser's `localStorage` as a stand-in for a real database.

This matters for how you read the rest of this document: numbers like "₦1,284,300 revenue" or "58 orders" are **seed/demo data**, not real transactions. Buttons that say "Add Product," "Approve," or "Suspend" genuinely update the page and (in the admin console) persist across reloads — but only inside your own browser, not on a server other people can see.

2. Tech stack & conventions

- **No build step, no framework.** Plain HTML files, plain CSS files, plain `<script>` tags. Open any `.html` file directly in a browser and it works.
 - **One shared design system per app area.** `customer/assets/style.css`, `vendor/assets/style.css`, and `admin/assets/style.css` each define the *same* base design tokens and layout primitives (sidebar, header, bottom nav, stat cards, tables, modals, forms, switches). This is intentional, established duplication — each area's stylesheet is self-contained so a change in one app doesn't silently break another, at the cost of some repeated CSS between them. The root `style.css` is a separate, simpler design system used only by the public marketing pages (`index.html`, `about.html`, `contact.html`, `signin.html`, `signup.html`).
 - **"App shell" pattern.** Every logged-in page (customer, vendor, admin) uses the same skeleton: a collapsible left sidebar on desktop ($\geq 900\text{px}$), a bottom tab bar on mobile, a dark header with the VETRA logo, and a `<main class="page-shell">` content area. `assets/interactions.js` (customer, vendor) and `assets/ui.js` (admin) wire up the sidebar collapse/expand behavior identically in each area.
 - **Mock data lives in JS.** Product lists, orders, and customer/vendor records are hard-coded into the HTML or into a `assets/data.js` -style file, not fetched from an API.
 - **Live chat.** Every page includes the Smartsupp live-chat snippet, with `hideWidget: true` so it only opens when a page's "Support" icon calls `smartsupp('chat:open')` (see `assets/support.js`, shared verbatim between customer and vendor).
-

3. Folder map

```
VETRA/
├── index.html, about.html, contact.html      Public marketing site
├── signin.html, signup.html                  Buyer/Vendor auth (mock)
├── style.css, signup.css                     Public-site design system
├── main.js, signin.js, signup.js             Public-site + auth page JS
├──
├── customer/                               Buyer-facing app
│   ├── dashboard.html, explore.html, category.html,
│   │   store.html, cart.html, orders.html, chat.html,
```

```

├── notifications.html, settings.html
├── assets/ (style.css, interactions.js, filters.js, orders.js, support.js)
├── vendor/                                Seller-facing app
│   ├── dashboard.html, products.html, orders.html,
│   ├── earnings.html, profile.html, notifications.html
│   └── assets/ (style.css, interactions.js, product-actions.js,
│               add-product.js, orders.js, order-tracking.js,
│               reports.js, profile.js, support.js)
├── extras/                                Retired/unused assets, kept but not referenced
│   └── logo.png, logo-mark.png, logo-mark-white.png, *-avatar-dummy.jpg
├── admin/                                NEW – staff-only console (this session)
│   ├── login.html                        Admin sign-in gate
│   ├── dashboard.html                   Platform overview
│   ├── customers.html                   Manage buyer accounts (list)
│   ├── customer-detail.html             One customer's full profile + activity
│   ├── vendors.html                     Manage vendor stores (list)
│   ├── vendor-detail.html               One vendor's full profile + activity
│   ├── reports.html                     Reports & disputes queue
│   ├── activity.html                    Audit trail / activity log
│   ├── settings.html                    Admin profile, team, platform switches
│   └── assets/ (style.css, data.js, ui.js, dashboard.js, customers.js,
│               customer-detail.js, vendors.js, vendor-detail.js,
│               reports.js, activity.js, settings.js)

```

4. Public site

Page	Purpose
index.html	Landing page — hero, featured categories/products, marketing content.
about.html	Brand story / mission.
contact.html	Contact info / form (static).
signin.html	Buyer/Vendor sign-in, toggled by a "Buyer / Vendor" tab. A small "Admin sign in" link at the bottom now routes staff to admin/login.html .
signup.html	Buyer/Vendor account creation, same toggle pattern as sign-in; supports a #Vendor URL hash to land directly on the vendor tab.

How sign-in "works" today: [signin.js](#) validates only that the visible fields aren't empty, then redirects — Buyer → [customer/dashboard.html](#) , Vendor → [vendor/dashboard.html](#) . No password is actually checked and no session is created. [admin/login.html](#) follows the same mock pattern, redirecting to [admin/dashboard.html](#) . **This is the single most important thing to replace before going live** — see §9.

5. Customer (buyer) app

All pages live in [customer/](#) and share the sidebar: **Dashboard, Explore, Categories, Cart, Chat, Notifications, Settings** (via the header/profile icon).

Page	What it does
dashboard.html	Home feed: promo banner carousel, category rail, featured product grid.

Page	What it does
<code>explore.html</code>	Full product browse grid.
<code>category.html</code>	Category grid + a filtered product view; <code>assets/filters.js</code> (<code>initProductFilters</code>) filters the visible product cards client-side by category/price and shows an empty state if nothing matches.
<code>store.html</code>	A single vendor's storefront (name, products, and a Reviews section — see below) — <code>id="store-name"</code> is filled in by JS from a query param/mock lookup.
<code>cart.html</code>	Cart line items, quantity steppers, delivery-option radios, and an order summary card with a "Checkout" button.
<code>orders.html</code>	New — order history and per-order delivery tracking. Status filter tabs (All/Pending/Processing/Shipped/Out for delivery/Delivered/Cancelled) via <code>wireOrderFilterTabs</code> ; each order card is also a click-to-expand accordion (<code>wireOrderExpand</code> , both in <code>assets/orders.js</code>) revealing a step timeline (Placed → Processing → Shipped → Out for delivery → Delivered) that mirrors the escrow flow described on <code>../buyer-protection.html</code> . Not in the main sidebar/bottom-nav (both are already at their intended item count) — reached from Settings → Account overview → "View orders" and from the "Order delivered" notification card on <code>notifications.html</code> .
<code>chat.html</code>	Buyer↔vendor messaging UI — conversation list + message thread, styled like a chat app (not connected to a real messaging backend).
<code>notifications.html</code>	List of notification cards (order updates, promos); the "Order delivered" card links to <code>orders.html</code> .
<code>settings.html</code>	Account & preference toggles (notifications, etc.); the "Orders placed" row under Account overview links to <code>orders.html</code> .

`customer/assets/interactions.js` wires: sidebar collapse/reopen, the banner carousel's dots, and `wireAddToCartButtons` (adds a fake line item / visual feedback when "Add to cart" is clicked — there's no real cart persistence between pages, since nothing here uses `localStorage`).

Reviews (`store.html`). Each mock vendor in the page's `VENDORS` object now carries a `reviews` array (name, rating, date, text), rendered as an average-score summary plus a review-card list, both tagged "Verified purchase" per the "verified reviews only" claim made elsewhere on the site. A "Write a review" form (star-picker + textarea) prepends a new review in-memory on submit — since there's no backend, it can't actually check for a completed order, so the form's hint text says as much and every submission is treated as if that check passed.

6. Vendor (seller) app

All pages live in `vendor/` and share the sidebar: **Dashboard, Products, Orders, Earnings, Profile**.

Page	What it does
<code>dashboard.html</code>	Store KPIs (revenue, orders, active listings, store views), a recent-orders list, and a "My Products" preview grid. Has the Add Product button, which opens a modal form.
<code>products.html</code>	Full product grid with Edit / Remove actions per card (<code>vendor/assets/products.js</code>).
<code>orders.html</code>	Order list with status filter tabs (Pending/Processing/Shipped/Out for delivery/Delivered/Cancelled) via <code>wireOrderFilterTabs</code> in <code>orders.js</code> . Each order has an Update shipment button opening a modal (<code>assets/order-tracking.js</code>) to set shipment status, carrier, and tracking number — updates that order row's status pill and tracking line in place. Below the order list, a Reports against your store panel (read-only — see below) shows reports filed against this vendor.
<code>earnings.html</code>	Revenue breakdown / earnings history.
<code>profile.html</code>	Store profile form (name, owner, email, phone, address, bio — starts read-only, "Edit Profile" unlocks the fields), notification toggles, security section (change password, 2FA, sign out), and a Danger Zone (deactivate store / delete account).

Page	What it does
notifications.html	Vendor-side notification feed.

Add Product modal ([vendor/assets/add-product.js](#)): a full form — name, category, price, a stock quantity stepper, description, up to 4 image upload slots with live preview, and one optional video upload slot — that builds a new product card and prepends it to the grid on submit ([handleSubmit](#) → [buildProductCard](#)). This is in-memory only: reloading the page loses any product you added, because (like the customer app) nothing here persists to [localStorage](#) .

Reports against your store ([orders.html](#) , [vendor/assets/reports.js](#)). A vendor can see reports filed against their store but can't resolve or dismiss them — that stays admin-only ([admin/reports.html](#)), keeping one place where a report is actually closed out. What a vendor *can* do is respond: an open report's **Submit evidence** button opens a modal (a response textarea plus a mocked photo-upload slot borrowed from the Add Product modal's media-upload UI) — submitting it swaps the button for an "Evidence submitted — awaiting review" tag on that card. This is deliberately a read-only mirror of the same report data admin sees, not a second copy of the moderation queue.

7. Admin console — new in this session

Why it was built

The request was for a place where VETRA staff can monitor customers and vendors platform-wide and suspend accounts, plus "other things that should be on the admin section." The console lives entirely under [admin/](#) , reuses the exact same app-shell/design system as the vendor app (so it feels native to the site, not bolted on), and — unlike the rest of the prototype — **persists its state to [localStorage](#)** under the key [vetra_admin_state_v1](#) , because moderation actions (suspending someone, resolving a report) need to survive a page reload to feel real. [admin/assets/data.js](#) is the single place that owns this state; every admin page reads and writes through its functions ([VetraAdmin.*](#)) instead of touching [localStorage](#) directly, so swapping in a real backend later means changing one file.

Getting in — and "who's logged in"

[admin/login.html](#) — a staff-only sign-in screen, styled like [signin.html](#) but with no Buyer/Vendor toggle. It's linked from the bottom of [signin.html](#) ("Vetra staff? Admin sign in"). Like the rest of the site's auth, it doesn't check a real password — but it does do one real thing: it looks the email you typed up against the admin team roster in [admin/assets/data.js](#) and, if it matches, tells the data layer **which admin is now "logged in"** ([VetraAdmin.setCurrentAdminByEmail\(\)](#)). That simulated session is what makes the rest of this section possible — every action taken anywhere in the console is attributed to whoever is currently "logged in," and the console's own role check (Super Admin vs. Moderator vs. Support) reads from it too. The sign-in page lists the three seeded demo accounts and their roles so you can try each one. See §9 for what a real version needs (this is still just an email lookup, not a password check).

Passwords, photos, and "who did what"

Three things were added on top of the original monitor/suspend request, all in direct response to follow-up asks in this session:

- **Reset Password (customers & vendors).** Every customer/vendor row (on [customers.html](#) / [vendors.html](#)) and every detail page has a **Reset Password** button. Confirming it generates a temporary password and shows it to the admin in a "share this with them securely" panel — there's no real account system to actually change a password against, so this is as far as a backend-less prototype can go, but the action is logged to that account's activity history like any other.
- **Admin profile photos.** Settings → Admin Profile has a real, working **Change Photo** upload (not a placeholder): it reads the chosen image with [FileReader](#) , stores it as a base64 data URL on that admin's team record, and immediately reflects it in the profile card, the header avatar on every admin page, and the Admin Team list. This works because the admin console already persists to [localStorage](#) , unlike the rest of the prototype.
- **Attribution — "which admin attended to a case."** Every activity log entry now records who performed it ([actorId](#) / [actorName](#)), and every report gets an [attendedBy](#) / [attendedAt](#) pair the moment it's resolved, dismissed, or resolved-via-suspend. That attribution shows up everywhere the relevant record appears — the activity feed ("by Adaeze Umeh"), a report card ("Attended by Adaeze Umeh"), and a customer/vendor's own activity history — regardless of which admin is viewing it. Case-level attribution is intentionally visible to every admin; it's the console-wide activity *browsing* below that's role-gated.

Pages

dashboard.html — **Platform overview**. Six KPI cards (Total Customers, Total Vendors, Platform Orders, Platform Revenue, Suspended Accounts, Open Reports), a **Quick Actions** row linking straight into a pre-filtered view of the other pages, a **Recent Activity** preview (last 6 audit-log entries), and a **Pending Vendor Applications** table where a new store's application can be **Approved** or **Rejected** right from the dashboard.

customers.html — **Manage buyers**. A searchable, filterable table of every customer (name/email, join date, order count, lifetime spend, status). Each row has **View** and either **Suspend** or **Reactivate** depending on current status. Suspending asks for an optional reason via a confirm modal — the reason is stored in the activity log so there's a record of *why*.

customer-detail.html?id=<id> — **One customer, in full**. Clicking a customer's name or **View** opens their complete profile instead of a quick-look popup: contact info (phone, delivery address), how and when they signed up, last login, lifetime order/spend stats, **any reports filed against their account** (pulled from the Reports queue), and their **entire activity history** — every suspend/reactivate action ever taken on this specific account, not just the platform-wide feed. Suspend/Reactivate works from here too, so a moderator reviewing an account doesn't have to bounce back to the list to act on what they just read.

vendors.html — **Manage stores**. Same shape as Customers, plus a **Pending Approval** state: new vendor applications show **Approve/Reject** instead of Suspend until an admin acts on them. Filter tabs: All / Active / Pending Approval / Suspended.

vendor-detail.html?id=<id> — **One store, in full**. Same idea as the customer detail page: store contact info, category, a short description, owner, join date, last login, products/orders/revenue stats, any reports filed against the store, and its complete activity history. The action buttons adapt to status — **Approve/Reject** while pending, **Suspend/Reactivate** once live.

reports.html — **Reports & disputes**. A moderation queue of flagged accounts, listings, and buyer↔vendor disputes (e.g. "item didn't match description," a chargeback dispute, a suspicious listing). Each open report can be **Marked Resolved**, **Dismissed**, or — if it's against a customer or vendor — resolved with a one-click **Suspend Account**, which suspends the account *and* marks the report resolved in a single confirmation.

activity.html — **Activity log / audit trail**. Every suspend, reactivate, approve, reject, resolve, dismiss, password reset, invite, and admin-removal action performed anywhere in the admin console is appended here automatically, newest first, with a relative timestamp ("2h ago"), a type tag, and — for anything an admin actually did — **who did it** ("by Adaeze Umeh"). Filterable by Account Actions / Vendor Approvals / Reports / Orders / Logins. This is what makes the console feel like a real moderation tool rather than a set of disconnected switches — every destructive action leaves a trace.

Visibility here depends on role: a **Super Admin** sees everyone's activity and gets a **"filter by admin"** dropdown to narrow it to one person; a **Moderator or Support** admin only sees platform events (orders, customer/vendor logins — nothing admin-specific) plus their *own* actions, with a note explaining that Super Admins can see more. This is enforced by `VetraAdmin.getVisibleActivity()`, which every activity view (this page and the dashboard's preview) calls instead of the raw log — but, like every other role check in this prototype, it's a client-side filter, not a real permission boundary (see §9).

settings.html — **Admin profile & platform controls**. - **My Profile**: reflects whoever actually signed in (name, email, role badge), with a working **Change Photo** upload. - **Platform Controls**: toggles for "require approval for new vendors," "auto-flag suspicious listings," "allow guest checkout" (**on by default** — buyers can check out without an account unless an admin turns it off here), "maintenance mode" (cosmetic switches — see §9). - **Admin Team**: the full list of who has console access and their role (Super Admin / Moderator / Support), each with their photo if they've set one, and a **Remove** action that goes through the shared confirm modal. Removing the platform's *last* Super Admin is blocked with an explanation, so the console can't be locked with zero full-access admins. - **Add Admin — invite + email verification**. Clicking **Add Admin** doesn't add someone immediately: it collects name/email/role and creates a *pending invitation*, then opens a **Verify Email** step. Since this prototype has no mail server to actually deliver anything, the code that "would have been emailed" is shown right in the UI, and — per how this was asked for — **entering anything in the verification field completes it**; this is a simulated verification step to demonstrate the invite→verify shape of a real onboarding flow, not a real code check (see `VetraAdmin.verifyTeamInvite()` in `admin/assets/data.js`, and §9). Pending invitations appear in their own list with **Verify/Cancel** actions until completed. - **Notifications** and **Security** sections matching the vendor profile page's pattern (password, 2FA, sign out). - **Data & Danger Zone → Reset Demo Data**: wipes the admin console's `localStorage` state and reloads the original seed data (team roster and any pending invites included). This exists *specifically because* this is a prototype with no real backend — it's how you get back to a clean demo state after clicking around, and it would not exist in a production build.

The shared confirm modal

Every destructive or state-changing action (suspend, approve, reject, resolve, dismiss, reset demo data) goes through **one reusable confirm dialog** (`AdminUI.confirm()` in `admin/assets/ui.js`), not a bare `window.confirm()`. It shows exactly

what's about to happen, names the target by name, optionally collects a reason, and only then calls into `VetraAdmin.*` to make the change. This was a deliberate choice: an admin console where "Suspend" fires immediately on click, with no confirmation, is a foot-gun.

What "add other things that should be on the admin section" became

Beyond the explicitly requested monitoring + suspend features, this session added: 1. **Vendor approval workflow** (pending → active/rejected) — without this, every new vendor signup would go live unmoderated. 2. **Reports & disputes queue** — a place for the "someone flagged this" workflow that any marketplace needs; suspending straight from a report closes the loop between "a problem was reported" and "an admin acted on it." 3. **Activity log / audit trail** — accountability for who did what, which also makes the console testable/demoable (you can see the effect of your own actions). 4. **Admin team/roles list** — a console with a hardcoded single admin user doesn't reflect that real platforms have multiple staff with different permission levels. 5. **Reset Demo Data** — a prototype-specific affordance so the demo doesn't get "stuck" in a suspended/modified state after someone tries the buttons. 6. **Admin login gate** (`admin/login.html`) — the original ask assumed an admin section existed on the site already reachable somehow; since it didn't, this is the entry point, reachable from `signin.html` . 7. **Full customer/vendor detail pages** (`customer-detail.html` , `vendor-detail.html`) — requested directly: monitoring "all their account activity and user data" needed more room than a quick-look popup could give, so each account now gets its own page with full profile info, a filtered activity history, and any reports against it. 8. **Add/Remove Admin** on Settings — also requested directly: the console needed a real way to grow (or shrink) its own staff list, not just display three hardcoded names. 9. **Password reset for customers and vendors** — requested directly: an admin console that can suspend an account but can't help someone locked out of it is missing a basic support tool. 10. **Admin profile photos** — requested directly, with a real (client-side) upload rather than a placeholder, since the console already has somewhere to persist it. 11. **Per-admin activity attribution + role-gated activity browsing** — requested directly ("let super admins see the activities of other admins... let all admins see which admin attended to a case"): every log entry and every report now records who acted on it, visible to all; but *browsing* the full log is Super-Admin-only, with everyone else scoped to platform events and their own actions. 12. **Admin invite + simulated email verification** — requested directly: adding a new admin now goes through an invite-and-verify step instead of adding them outright, standing in for the real "confirm your email" step a production onboarding flow would have. 13. **Guest checkout, on by default** — requested directly: `admin/settings.html` 's "Allow guest checkout" toggle now starts checked, and `signin.html` gained a "Continue as Guest →" link (buyer tab only) straight into `customer/dashboard.html` , so the platform-control setting has a matching entry point on the actual sign-in page.

8. Site-wide review: redundancies, fixes, and what still needs attention

An audit pass was run across the public site, customer app, and vendor app (the admin section was excluded from the audit since it was just built and independently verified separately — every stat card, table, filter tab, search box, and confirm-modal flow on all six admin pages was clicked through in an automated browser test and confirmed to work, including that suspending a customer persists after a page reload). Everything below that was flagged as an actual bug was fixed in this session and re-verified with another automated browser pass (checked live: cart quantity/price/subtotal math, "Contact vendor" navigation, the checkout flow, the chat Send button, both settings buttons, the vendor product grid after the JS consolidation, and the signup→signin vendor-mode handoff — zero console errors).

Fixed

Where	Problem	Fix
<code>customer/cart.html</code>	The Checkout , Save for later , quantity −/+ steppers, and Contact vendor buttons had no JS behind them at all — clicking any of them did nothing. This is the site's core purchase flow, so it was the highest-impact finding.	Added <code>customer/assets/cart.js</code> : quantity steppers now recompute that line's price and the whole order summary (subtotal + total) live; "Contact vendor" opens the chat app; "Checkout"/"Save for later" give real click feedback (a processing state, then a confirmation) in place of the mock payment/save API this prototype doesn't have.

Where	Problem	Fix
<code>customer/chat.html</code>	The Send button (and pressing Enter) did nothing — messages could never actually be sent in the demo chat UI.	Wired <code>sendMessage()</code> into the chat-window render: it appends the message to that conversation's history, renders it into the thread, updates the sidebar preview/time, and clears the input.
<code>customer/settings.html</code>	" Manage payment methods " and " Deactivate account " had no <code>id</code> and no handler — every other button on that page was wired, these two were an apparent oversight.	Gave both an id and a handler matching the same confirm+aler "hook this up to your API" placeholder pattern already used by <code>vendor/assets/profile.js</code> danger-zone buttons, so the whole site's mock-action convention now applies consistently here too.
<code>signup.js</code>	After validating the form, the Continue button just sat on "Creating account..." forever — the signup flow had no redirect at all, unlike sign-in.	Added the same delayed-redirect pattern <code>signin.js</code> uses, sending the new account to <code>signin.html</code> (or <code>signin.html#Vendor</code> if they registered as a vendor).
<code>signin.js</code>	Didn't handle a <code>#Vendor</code> URL hash, even though <code>signup.js</code> already produced/expected that pattern for its own hash-based deep link — a latent inconsistency that the signup fix above would otherwise have silently tripped over (landing on the Buyer tab after registering as a Vendor).	Added the matching hash check so <code>signin.html#Vendor</code> now pre-selects the Vendor tab, closing the gap between the two auth pages.
<code>vendor/assets/dashboard.js</code> + <code>vendor/assets/products.js</code>	Both files contained byte-for-byte identical <code>wireVendorProductActions()</code> / <code>wireAddProductButton()</code> functions — accidental copy-paste, not the intentional shared-CSS convention used elsewhere, so a future edit to one copy could silently drift from the other.	Merged both into one <code>vendor/assets/product-actions.js</code> , updated <code>dashboard.html</code> and <code>products.html</code> to load it, deleted the two duplicates. Re-verified Edit/Remove and the Add Product modal still work on both pages.

Reviewed and left as-is (working as intended)

- **Sidebar/bottom-nav links** across every customer and vendor page resolve correctly and are consistent sibling-to-sibling.
- `vendor/assets/add-product.js` modal (shared by `dashboard.html` and `products.html`) opens/closes correctly via its close button, Cancel, backdrop click, and Escape.
- **Contact form, FAQ accordion** (`script.js`) on the public pages work.
- Several buttons intentionally `alert()` a "hook this up" placeholder rather than navigate (vendor Edit product, avatar edit, change password) — this is the established, deliberate pattern for "the backend doesn't exist yet" across the codebase, not a bug, so these were left alone.
- `customer/assets/interactions.js` vs `vendor/assets/interactions.js` and `signin.js` vs `signup.js` 's buyer/vendor toggle logic are near-duplicates of each other by the same intentional per-area convention described in §2/§6 — low-priority, cosmetic overlap, not touched.

Still open (needs a human decision, not a quick fix)

- `customer/cart.html` 's "Contact vendor" opens the chat list rather than a specific conversation, because cart line items don't currently carry a vendor id that maps to `chat.html` 's mock conversation data. Once products/cart items carry a

real vendor id, point this at `chat.html?chat=<id>` the same way `store.html` already does.

- Everything in §9 below (auth, database, uploads, cart/chat persistence) — these are architecture-level gaps, not bugs in the existing code.

Feature gaps closed by a follow-up audit

A later audit pass (live-rendering check across 15 priority pages at mobile and desktop widths, plus a static sweep for orphaned images and dead JS) found zero rendering bugs but three gaps between what the site *claims* and what it actually lets someone do — all three closed in this session:

1. **No order history or delivery tracking**, despite "real-time delivery tracking" being advertised on `index.html` and `buyer-protection.html`, and vendors being told to "log tracking details" on `vendor-protection.html`. Added `customer/orders.html` (order history + expandable tracking timeline) and an **Update shipment** modal on `vendor/orders.html` (status/carrier/tracking-number, per order) — see §5 and §6 above.
2. **No customer-facing review UI**, despite "verified reviews only" being a repeated selling point. Added a Reviews section (summary + review list + write-a-review form) to `customer/store.html` — see §5 above.
3. **No vendor-facing view of reports filed against their store** — admin had a full moderation queue (`admin/reports.html`) but a vendor had no way to even see a report existed, let alone respond to it, despite `vendor-protection.html` walking vendors through exactly that flow. Added a read-only **Reports against your store** panel to `vendor/orders.html` — see §6 above.

The same audit also found 5 image files with zero references anywhere in the codebase (the original `logo.png` before the brand-asset generation pass, two unused mark-only logo variants, and two duplicate `avatar-dummy.jpg` files shadowed by an identically-purposed `.png`). Moved to `extras/` rather than deleted outright, in case they're wanted later.

9. What a real backend needs to replace

This prototype intentionally fakes several things. Before any of this touches real users:

1. **Authentication.** `signin.js`, `signup.js`, and `admin/login.html`'s inline script all just check "is this field non-empty" (`admin/login.html` additionally matches the email against the team roster to know *which* admin it is, but never checks a password) and redirect. There is no real password check, no session/token, and — critically — **no server-side check that a signed-in admin is actually an admin**. Anyone who knows the URL can open `admin/dashboard.html` directly and, via `localStorage`, even set `currentAdminId` to any admin's id. A real build needs real auth (e.g. sessions or JWTs) with role-based access control gating every `admin/*.html` route server-side, not just a client-side lookup.
2. **Admin roles gate almost nothing server-side.** The one real enforcement right now is client-side: `getVisibleActivity()` filters what a non-Super-Admin's activity log shows, and `removeTeamMember()` blocks deleting the last Super Admin. Nothing stops a Moderator or Support admin from calling `setVendorStatus`, `resetCustomerPassword`, etc. directly — the UI just doesn't currently offer every action to every role by design intent, not by enforcement. Once real auth exists, every `VetraAdmin.set*` / `reset*` / `add*` / `remove*` call needs a real permission check behind it.
3. **The admin invite/verification flow is fully simulated.** `inviteTeamMember()` generates a code and displays it directly in the UI instead of emailing it, and `verifyTeamInvite()` accepts *any* non-empty value as correct — there is no real email delivery or code matching. A real build needs an actual mail provider and a code comparison that can fail.
4. **Password reset doesn't reset anything real.** `resetCustomerPassword()` / `resetVendorPassword()` generate a temporary string and show it to the admin — there's no real customer/vendor password store to write it to, since buyer/vendor accounts aren't real accounts in this prototype either (see point 1). A real build needs this to actually invalidate the old password and email/notify the account holder, not display the new one to the admin.
5. **A real database.** Every "list of customers/vendors/products/orders" is hard-coded JS or, in the admin console, `localStorage` — which is per-browser and never seen by anyone else. All of it needs to move to a real database behind an API.
6. **admin/assets/data.js is the seam.** It already isolates every read/write behind named functions (`getCustomers`, `setCustomerStatus`, `resetCustomerPassword`, `getVendors`, `setVendorStatus`, `resetVendorPassword`, `getReports`, `setReportStatus`, `getActivity`, `getVisibleActivity`, `logActivity`, `getTeam`, `setTeamMemberAvatar`, `removeTeamMember`, `inviteTeamMember`, `verifyTeamInvite`, `getCurrentAdmin`, `setCurrentAdminByEmail`, `getStats`, plus the `getActivityForTarget` / `getReportsForTarget` lookups the detail

pages use). Swapping its internals from `localStorage` to `fetch()` calls against a real API — and its avatar storage from base64 data URLs to real file uploads — is the intended upgrade path; no other admin file talks to storage directly.

7. **Platform-control toggles** on `admin/settings.html` (require vendor approval, auto-flag listings, guest checkout, maintenance mode) are currently cosmetic switches with no effect — they need to be wired to real server-side flags that other parts of the app actually check.
8. **File uploads** (product images/video in the vendor "Add Product" modal, avatar edit buttons) only preview locally in the browser; nothing is actually uploaded anywhere.
9. **Cart and chat** (`customer/cart.html` , `customer/chat.html`) are UI-only — no real cart persists across pages, and no real messages are sent.

10. Design system reference (for anyone extending a page)

Shared building blocks used across customer, vendor, and admin (defined per-area in each `assets/style.css` , kept visually identical):

- **Layout:** `.app-shell` → `.sidebar` (desktop) + `.phone` (scroll container) → `header#app-header` → `main.page-shell` → `nav.bottom-nav` (mobile).
- **Cards/sections:** `.vendor-section` (generic card container, used well beyond vendor pages), `.stat-grid` / `.stat-card` (KPI tiles).
- **Tables (admin only):** `.table-wrap` + `table.data-table` , row actions via `.table-actions button` with semantic classes `.btn-view/.btn-suspend/.btn-activate/.btn-approve/.btn-reject` .
- **Status pills:** `.status-pill` (orders: pending/processing/shipped/out-for-delivery/completed/cancelled — the last two states added for order tracking), `.badge` (admin + vendor reports panel: active/suspended/pending/resolved/dismissed/open).
- **Forms:** `.form-grid` / `.form-group` / `.form-input` / `.form-textarea` , `.switch` (toggle), `.danger-zone` (destructive settings). `customer/assets/style.css` also defines a lighter `.field-group` (label + input/select/textarea) shared by `settings.html` and `store.html` 's review form.
- **Modals:** `.modal-overlay` + `.modal-panel` (Add Product, confirm dialogs, detail views, vendor/orders.html's Update Shipment and Submit Evidence modals).
- **Filter tabs:** `.order-filter-tabs` `.filter-tab.active` .
- **Report cards:** `.report-list` / `.report-card` / `.report-card-head` / `.report-reason` / `.report-meta-row` / `.report-actions` — originally admin-only (`admin/assets/style.css`), the same class names/shapes now also exist in `vendor/assets/style.css` for the read-only vendor-side view, so a report renders identically in both places.

If you add a new page to any app, copy the closest existing sibling page's `<head>` /sidebar/header/bottom-nav markup rather than writing it from scratch — this is how every existing page in the codebase was built, and it's what keeps the three apps visually consistent.